

Table of Contents

1 Overview	3
1.1 Revision History	3
1.2 Purpose	3
1.3 Status Code Categories	3
1.3.1 1xx: Informational	3
1.3.2 2xx: Success	4
1.3.3 3xx: Redirection	4
1.3.4 4xx: Client Error	5
1.3.5 5xx: Server Error	8
1.4 REST Specific HTTP Status Codes	8
1.4.1 200 (OK)	9
1.4.2 201 (Created)	9
1.4.3 202 (Accepted)	9
1.4.4 204 (No Content)	9
1.4.5 301 (Moved Permanently)	10
1.4.6 302 (Found)	10
1.4.7 303 (See Other)	10
1.4.8 304 (Not Modified)	10
1.4.9 307 (Temporary Redirect)	11
1.4.10 400 (Bad Request)	11
1.4.11 401 (Unauthorized)	11
1.4.12 403 (Forbidden)	12
1.4.13 404 (Not Found)	12
1.4.14 405 (Method Not Allowed)	12
1.4.15 406 (Not Acceptable)	12
1.4.16 412 (Precondition Failed)	12
1.4.17 415 (Unsupported Media Type)	13
1.4.18 500 (Internal Server Error)	13
1.4.19 501 (Not Implemented)	13
2 References	14
3 Glossary	14
3.1 Acronyms	14

1 Overview

1.1 Revision History

Date	Comment	Author(s)
09/26/2024	Initial Draft Version 1.0	Rohtash Lakra

1.2 Purpose

REST APIs use the **Status-Line** part of an HTTP response message to inform clients of their request's overarching result. [RFC 2616](#) defines the [Status-Line syntax](#) as shown below:

Status-Line = HTTP-Version SP Status-Code SP Reason-Phrase CRLF

HTTP defines these standard status codes that can be used to convey the results of a client's request.

1.3 Status Code Categories

The status codes are divided into five categories.

1.3.1 1xx: Informational

Communicates transfer protocol-level information.

Status Code	Description
100 Continue	An interim response. Indicates to the client that the initial part of the request has been received and has not yet been rejected by the server. The client SHOULD continue by sending the remainder of the request or, if the request has already been completed, ignore this response. The server MUST send a final response after the request has been completed.
101 Switching Protocol	Sent in response to an Upgrade request header from the client and indicates the protocol the server is switching to.
102 Processing (WebDAV)	Indicates that the server has received and is processing the request, but no response is available yet.
103 Early Hints	Primarily intended to be used with the Link header. It suggests the user agent start preloading the resources while the server prepares a final response.

HTTP Status Codes

1.3.2 2xx: Success

Indicates that the client's request was accepted successfully.

Status Code	Description
200 OK	Indicates that the request has succeeded.
201 Created	Indicates that the request has succeeded, and a new resource has been created as a result.
202 Accepted	Indicates that the request has been received but not completed yet. It is typically used in log running requests and batch processing.
203 Non-Authoritative Information	Indicates that the returned metainformation in the entity header is not the definitive set as available from the origin server but is gathered from a local or third-party copy. The set presented MAY be a subset or superset of the original version.
204 No Content	The server has fulfilled the request but does not need to return a response body. The server may return the updated meta information.
205 Reset Content	Indicates the client to reset the document that sent this request.
206 Partial Content	It is used when the Range header is sent from the client to request only part of a resource.
207 Multi-Status (WebDAV)	An indicator to a client that multiple operations happened, and that the status for each operation can be found in the body of the response.
208 Already Reported (WebDAV)	Allows a client to tell the server that the same resource (with the same binding) was mentioned earlier. It never appears as a true HTTP response code in the status line, and only appears in bodies.
226 IM Used	The server has fulfilled a GET request for the resource, and the response is a representation of the result of one or more instance-manipulations applied to the current instance.

1.3.3 3xx: Redirection

Indicates that the client must take some additional action in order to complete their request.

Status Code	Description
300 Multiple Choices	The request has more than one possible response. The user-agent or user should choose one of them.

HTTP Status Codes

301 Moved Permanently	The URL of the requested resource has been changed permanently. The new URL is given by the Location header field in the response. This response is cacheable unless indicated otherwise.
302 Found	The URL of the requested resource has been changed temporarily. The new URL is given by the Location field in the response. This response is only cacheable if indicated by a Cache-Control or Expires header field.
303 See Other	The response can be found under a different URI and SHOULD be retrieved using a GET method on that resource.
304 Not Modified	Indicates the client that the response has not been modified, so the client can continue to use the same cached version of the response.
305 Use Proxy (Deprecated)	Indicates that a requested response must be accessed by a proxy.
306 (Unused)	It is a reserved status code and is not used anymore.
307 Temporary Redirect	Indicates the client to get the requested resource at another URI with same method that was used in the prior request. It is similar to 302 Found with one exception that the same HTTP method will be used that was used in the prior request.
308 Permanent Redirect (experimental)	Indicates that the resource is now permanently located at another URI, specified by the Location header. It is similar to 301 Moved Permanently with one exception that the same HTTP method will be used that was used in the prior request.

1.3.4 4xx: Client Error

This category of error status codes points the finger at clients.

Status Code	Description
400 Bad Request	The server could not understand the request due to incorrect syntax. The client should NOT repeat the request without modifications.
401 Unauthorized	Indicates that the request requires user authentication information. The client MAY repeat the request with a suitable Authorization header field
402 Payment Required (Experimental)	Reserved for future use. It is aimed for use in the digital payment systems.

HTTP Status Codes

403 Forbidden	Unauthorized request. The client does not have access rights to the content. Unlike 401, the client's identity is known to the server.
404 Not Found	The server can not find the requested resource.
405 Method Not Allowed	The server knows the request HTTP method, but it has been disabled and cannot be used for that resource.
406 Not Acceptable	The server doesn't find any content that conforms to the criteria given by the user agent in the Accept header sent in the request.
407 Proxy Authentication Required	Indicates that the client must first authenticate itself with the proxy.
408 Request Timeout	Indicates that the server did not receive a complete request from the client within the server's allotted timeout period.
409 Conflict	The request could not be completed due to a conflict with the current state of the resource.
410 Gone	The requested resource is no longer available on the server.
411 Length Required	The server refuses to accept the request without a defined Content-Length. The client MAY repeat the request if it adds a valid Content-Length header field.
412 Precondition Failed	The client has indicated preconditions in its headers that the server does not meet.
413 Request Entity Too Large	The request entity is larger than the limits defined by the server.
414 Request-URI Too Long	The URI requested by the client is longer than the server can interpret.
415 Unsupported Media Type	The media type in Content-type of the request is not supported by the server.
416 Requested Range Not Satisfiable	The range specified by the Range header field in the request can't be fulfilled.
417 Expectation Failed	The server can't meet the expectations indicated by the request header field.

HTTP Status Codes

418 I'm a teapot (RFC 2324)	It was defined as an April Fool joke and is not expected to be implemented by actual HTTP servers. (RFC 2324). Some websites/providers use this response for requests they do not wish to handle, such as automated queries.
420 Enhance Your Calm (Twitter)	Returned by the Twitter Search and Trends API when the client is being rate-limited.
422 Unprocessable Entity (WebDAV)	The server understands the content type and syntax of the request entity, but it is still unable to process the request for some reason.
423 Locked (WebDAV)	The resource that is being accessed is locked.
424 Failed Dependency (WebDAV)	The request failed due to the failure of a previous request.
425 Too Early (WebDAV)	Indicates that the server is unwilling to risk processing a request that might be replayed.
426 Upgrade Required	The server refuses to perform the request. The server will process the request after the client upgrades to a different protocol.
428 Precondition Required	The origin server requires the request to be conditional.
429 Too Many Requests	The user has sent too many requests in a given amount of time ("rate limiting").
431 Request Header Fields Too Large	The server is unwilling to process the request because its header fields are too large.
444 No Response (Nginx)	The Nginx server returns no information to the client and closes the connection.
449 Retry With (Microsoft)	The request should be retried after performing the appropriate action.
450 Blocked by Windows Parental Controls (Microsoft)	Windows Parental Controls are turned on and are blocking access to the given webpage.
451 Unavailable For Legal Reasons	The user-agent requested a resource that cannot legally be provided.
499 Client Closed Request (Nginx)	The connection is closed by the client while HTTP server is processing its request, making the server unable to send the HTTP header back.

HTTP Status Codes

1.3.5 5xx: Server Error

The server takes responsibility for these error status codes.

Status Code	Description
500 Internal Server Error	The server encountered an unexpected condition that prevented it from fulfilling the request.
501 Not Implemented	The HTTP method is not supported by the server and cannot be handled.
502 Bad Gateway	The server got an invalid response while working as a gateway to get the response needed to handle the request.
503 Service Unavailable	The server is not ready to handle the request.
504 Gateway Timeout	The server is acting as a gateway and cannot get a response in time for a request.
505 HTTP Version Not Supported (Experimental)	The HTTP version used in the request is not supported by the server.
506 Variant Also Negotiates (Experimental)	Indicates that the server has an internal configuration error: the chosen variant resource is configured to engage in transparent content negotiation itself, and is therefore not a proper endpoint in the negotiation process.
507 Insufficient Storage (WebDAV)	The method could not be performed on the resource because the server is unable to store the representation needed to complete the request successfully.
508 Loop Detected (WebDAV)	The server detected an infinite loop while processing the request.
510 Not Extended	Further extensions to the request are required for the server to fulfill it.
511 Network Authentication Required	Indicates that the client needs to authenticate to gain network access.

1.4 REST Specific HTTP Status Codes

1.4.1 200 (OK)

It indicates that the REST API successfully carried out the client's requested action and that no more specific code in the 2xx series is appropriate.

Unlike the 204 status code, a 200 response should include a response body. The information returned with the response is dependent on the method used in the request, for example:

- GET an entity corresponding to the requested resource is sent in the response;
- HEAD the entity-header fields corresponding to the requested resource are sent in the response without any message-body;
- POST an entity describing or containing the result of the action;
- TRACE an entity containing the request message as received by the end server.

1.4.2 201 (Created)

A REST API responds with the 201 status code whenever a resource is created inside a collection. There may also be times when a new resource is created as a result of some controller action, in which case 201 would also be an appropriate response.

The newly created resource can be referenced by the URI(s) returned in the entity of the response, with the most specific URI for the resource given by a Location header field.

The origin server **MUST** create the resource before returning the 201 status code. If the action cannot be carried out immediately, the server **SHOULD** respond with a 202 (Accepted) response instead.

1.4.3 202 (Accepted)

A 202 response is typically used for actions that take a long while to process. It indicates that the request has been accepted for processing, but the processing has not been completed. The request might or might not be eventually acted upon, or even maybe disallowed when processing occurs.

Its purpose is to allow a server to accept a request for some other process (perhaps a batch-oriented process that is only run once per day) without requiring that the user agent's connection to the server persist until the process is completed.

The entity returned with this response **SHOULD** include an indication of the request's current status and either a pointer to a status monitor (job queue location) or some estimate of when the user can expect the request to be fulfilled.

1.4.4 204 (No Content)

The 204 status code is usually sent out in response to a PUT, POST, or DELETE request when the REST API declines to send back any status message or representation in the response message's body.

An API may also send 204 in conjunction with a GET request to indicate that the requested resource exists but has no state representation to include in the body.

If the client is a user agent, it **SHOULD NOT** change its document view from that which caused the request to be sent. This response is primarily intended to allow input for actions to take place without causing a change to the user agent's active document view. However, any new or updated metainformation **SHOULD** be applied to the document currently in the user agent's dynamic view.

The 204 response **MUST NOT** include a message body and is thus always terminated by the first empty line after the header fields.

1.4.5 301 (Moved Permanently)

The 301 status code indicates that the REST API's resource model has been significantly redesigned, and a new permanent URI has been assigned to the client's requested resource. The REST API should specify the new URI in the response's Location header, and all future requests should be directed to the given URI.

You will hardly use this response code in your API as you can always use the API versioning for the new API while retaining the old one.

1.4.6 302 (Found)

The HTTP response status code 302 Found is a common URL redirection method. An HTTP response with this status code will additionally provide a URL in the Location header field. The user agent (e.g., a web browser) is invited by a response with this code to make a second. Otherwise, an identical request to the new URL specified in the location field.

Many web browsers implemented this code in a manner that violated this standard, changing the request type of the new request to GET, regardless of the type employed in the original request (e.g., POST). RFC 1945 and RFC 2068 specify that the client is not allowed to change the method on the redirected request. The status codes 303 and 307 have been added for servers that wish to make unambiguously clear what kind of reaction the client expects.

1.4.7 303 (See Other)

A 303 response indicates that a controller resource has finished its work. Instead of sending a potentially unwanted response body, it sends the client the URI of a response resource. The response can be the URI of the temporary status message or the URI of some already existing, more permanent resource.

Generally speaking, the 303 status code allows a REST API to send a reference to a resource without forcing the client to download its state. Instead, the client may send a GET request to the value of the Location header.

The 303 response **MUST NOT** be cached, but the response to the second (redirected) request might be cacheable.

1.4.8 304 (Not Modified)

This status code is similar to 204 ("No Content") in that the response body must be empty. The critical distinction is that 204 is used when nothing is to be sent in the body, whereas 304

is used when the resource has not been modified since the version specified by the request headers If-Modified-Since or If-None-Match.

In such a case, there is no need to retransmit the resource since the client still has a previously downloaded copy.

Using this saves bandwidth and reprocessing on both the server and client, as only the header data must be sent and received in comparison to the entirety of the page being re-processed by the server, then sent again using more bandwidth of the server and client.

1.4.9 307 (Temporary Redirect)

A 307 response indicates that the REST API is not going to process the client's request. Instead, the client should resubmit the request to the URI specified by the response message's Location header. However, future requests should still use the original URI.

A REST API can use this status code to assign a temporary URI to the client's requested resource. For example, a 307 response can be used to shift a client request over to another host.

The Location field SHOULD give the temporary URI in the response. Unless the request method was HEAD, the entity of the response SHOULD contain a short hypertext note with a hyperlink to the new URI(s). If the 307 status code is received in response to a request other than GET or HEAD, the user agent MUST NOT automatically redirect the request unless the user can confirm it since this might change the conditions under which the request was issued.

1.4.10 400 (Bad Request)

400 is the generic client-side error status, used when no other 4xx error code is appropriate. Errors can be malformed request syntax, invalid request message parameters, deceptive request routing, etc.

The client SHOULD NOT repeat the request without modifications.

1.4.11 401 (Unauthorized)

A 401 error response indicates that the client tried to operate on a protected resource without providing the proper authorization. It may have provided the wrong credentials or none at all. The response must include a WWW-Authenticate header field containing a challenge applicable to the requested resource.

The client MAY repeat the request with a suitable Authorization header field. If the request already included Authorization credentials, then the 401 response indicates that authorization has been refused for those credentials. If the 401 response contains the same challenge as the prior response, and the user agent has already attempted authentication at least once, then the user SHOULD be presented the entity that was given in the response since that entity might include relevant diagnostic information.

1.4.12 403 (Forbidden)

A 403 error response indicates that the client's request is formed correctly, but the REST API refuses to honor it, i.e., the user does not have the necessary permissions for the resource. A 403 response is not a case of insufficient client credentials; that would be 401 ("Unauthorized").

Authentication will not help, and the request **SHOULD NOT** be repeated. Unlike a 401 Unauthorized response, authenticating will make no difference.

1.4.13 404 (Not Found)

The 404 error status code indicates that the REST API can't map the client's URI to a resource but may be available in the future. Subsequent requests by the client are permissible.

No indication is given of whether the condition is temporary or permanent. The 410 (Gone) status code **SHOULD** be used if the server knows, through some internally configurable mechanism, that an old resource is permanently unavailable and has no forwarding address. This status code is commonly used when the server does not wish to reveal exactly why the request has been refused or when no other response is applicable.

1.4.14 405 (Method Not Allowed)

The API responds with a 405 error to indicate that the client tried to use an HTTP method that the resource does not allow. For instance, a read-only resource could support only GET and HEAD, while a controller resource might allow GET and POST, but not PUT or DELETE.

A 405 response must include the Allow header, which lists the HTTP methods that the resource supports. For example:

Allow: GET, POST

1.4.15 406 (Not Acceptable)

The 406 error response indicates that the API is not able to generate any of the client's preferred media types, as indicated by the Accept request header. For example, a client request for data formatted as application/xml will receive a 406 response if the API is only willing to format data as application/json.

If the response is unacceptable, a user agent **SHOULD** temporarily stop receiving more data and query the user for a decision on further actions.

1.4.16 412 (Precondition Failed)

The 412 error response indicates that the client specified one or more preconditions in its request headers, effectively telling the REST API to carry out its request only if certain conditions were met. A 412 response indicates that those conditions were not met, so the API sends this status code instead of carrying out the request.

1.4.17 415 (Unsupported Media Type)

The 415 error response indicates that the API is not able to process the client's supplied media type, as indicated by the Content-Type request header. For example, a client request including data formatted as application/xml will receive a 415 response if the API is only willing to process data formatted as application/json.

For example, the client uploads an image as image/svg+xml, but the server requires that images use a different format.

1.4.18 500 (Internal Server Error)

500 is the generic REST API error response. Most web frameworks automatically respond with this response status code whenever they execute some request handler code that raises an exception.

A 500 error is never the client's fault, and therefore, it is reasonable for the client to retry the same request that triggered this response and hope to get a different response.

The API response is the generic error message, given when an unexpected condition was encountered and no more specific message is suitable.

1.4.19 501 (Not Implemented)

The server either does not recognize the request method, or it cannot fulfill the request. Usually, this implies future availability (e.g., a new feature of a web-service API).

2 References

- [Hypertext Transfer Protocol](#)
- [HTTP Status Codes](#)
- [REST HTTP Status Codes](#)

3 Glossary

Glossary of Terms, and Acronyms

3.1 Acronyms

Acronyms	Description
HTTP	Hypertext Transfer Protocol